AI MOVIE RECOMMENDATION SYSTEM 🎬

(Project 3)

Project Title

AI Movie Recommendation System Using Python

Submitted in Partial Fulfillment of the Requirements for the

DecodeLabs Artificial Intelligence Internship Program

Submitted By:

Priya Das

Internship Batch: June 2026

Submitted To:

DecodeLabs

Artificial Intelligence Internship Program

CONTENTS

PROJECT OVERVIEW

The AI Movie Recommendation System is a Python-based application designed to recommend movies according to the user's interests and preferences. The system takes the user's favorite movie genres as input and analyzes them using similarity matching logic to identify movies that best match the selected preferences.

The main objective of this project is to demonstrate the basic concepts of recommendation systems, which are widely used in modern applications such as Netflix, Amazon, and YouTube. The recommendation process is based on comparing user-selected genres with the genres associated with each movie in the dataset and calculating a similarity score.

The system displays personalized movie recommendations along with similarity scores, match percentages, and matched genres to help users understand why a particular movie was suggested. In addition, the application includes features such as recommendation history management, viewing previous recommendations, and clearing stored history through a user-friendly menu-driven interface.

This project provides practical experience in Python programming, data structures, pattern matching, file handling, and recommendation system concepts. It serves as an introductory implementation of AI-based recommendation logic and helps build a strong foundation for developing more advanced recommendation systems in the future.

OBJECTIVES

The main objectives of this project are:

• To develop a simple AI-based movie recommendation system using Python.

• To collect and process user preferences in the form of movie genres.

• To implement similarity matching logic for identifying relevant movies.

• To calculate similarity scores and match percentages between user preferences and movie genres.

• To provide personalized movie recommendations based on user interests.

• To maintain recommendation history using file handling techniques.

• To provide options for viewing and clearing recommendation history through a menu-driven interface.

• To understand the fundamental concepts of recommendation systems, pattern matching, and user preference analysis.

• To enhance Python programming skills by applying concepts such as functions, loops, lists, dictionaries, conditional statements, and file handling.

• To build a foundation for developing more advanced AI-powered recommendation systems in the future.

DATASET DESCRIPTION

The dataset used in this project is a manually created movie dataset stored in the form of a Python dictionary. Each movie is associated with one or more genres, which are used to match the user's preferences and generate recommendations.

The dataset contains a collection of popular movies from different genres such as Action, Sci-Fi, Adventure, Drama, Thriller, Romance, Fantasy, and Crime. Each movie title serves as a key, while the corresponding genres are stored as values in a list format.

Sample Dataset:

```
{
   "Interstellar": ["Sci-Fi", "Adventure", "Drama"],
   "Avengers Endgame": ["Action", "Adventure", "Sci-Fi"],
   "John Wick": ["Action", "Thriller"],
   "The Notebook": ["Romance", "Drama"],
   "Inception": ["Sci-Fi", "Action", "Thriller"]
}
```

Dataset Characteristics:

• Dataset Type: Structured Dataset

• Storage Format: Python Dictionary

• Number of Movies: 10

• Number of Genres: 8

• Genres Included:
- Action
- Sci-Fi
- Adventure
- Drama
- Thriller
- Romance
- Fantasy
- Crime

The dataset serves as the knowledge base of the recommendation system. The recommendation logic compares the genres selected by the user with the genres assigned to each movie and calculates similarity scores to identify the most relevant recommendations.

METHODOLOGY / IMPLEMENTATION

The AI Movie Recommendation System was developed using Python and follows a content-based recommendation approach. The system recommends movies by comparing user-selected genres with the genres associated with movies stored in the dataset.

Step 1: Dataset Creation

A movie dataset was created using a Python dictionary. Each movie is mapped to one or more genres, which serve as the basis for recommendation generation.

Step 2: User Preference Collection

The system prompts the user to enter their favorite movie genres in a comma-separated format. The input is processed and converted into a list of genres for further analysis.

Example:

Input:
Action, Sci-Fi

Processed Output:
["Action", "Sci-Fi"]

Step 3: Similarity Matching

The recommendation engine compares the user's preferred genres with the genres of each movie in the dataset. Set operations are used to identify common genres between the user preferences and movie genres.

Step 4: Similarity Score Calculation

A similarity score is calculated based on the number of matching genres. Movies with a higher number of matching genres receive higher scores and are considered more relevant.

Formula:

Similarity Score = Number of Common Genres

Step 5: Match Percentage Calculation

The system calculates a match percentage to indicate how closely a movie matches the user's preferences.

Formula:

Match Percentage = (Similarity Score / Total User Preferences) × 100

Step 6: Recommendation Ranking

All movies with at least one matching genre are stored and sorted in descending order based on their similarity scores. The top recommendations are then displayed to the user.

Step 7: Displaying Recommendations

The system displays:
• Movie Name
• Matched Genres
• Similarity Score
• Match Percentage

This helps users understand why a particular movie was recommended.

Step 8: Recommendation History Management

The generated recommendations and user ratings are stored in a text file named history.txt. This allows users to view previous recommendations and maintain a history of interactions.

Step 9: Menu-Driven Interface

A menu-driven system was implemented with the following options:

1. Get Recommendations
2. View History
3. Clear History
4. Exit

This improves usability and allows users to interact with the application efficiently.

Overall Workflow:

User Input
↓
Genre Processing
↓
Similarity Matching
↓
Score Calculation
↓
Recommendation Ranking
↓
Display Recommendations
↓
Save History

RESULTS AND ANALYSIS

The AI Movie Recommendation System was successfully developed and tested using different user preferences. The system accurately analyzed user-selected genres, calculated similarity scores, and generated relevant movie recommendations based on genre matching.

When a user entered preferred genres such as "Action" and "Sci-Fi", the system compared these preferences with the genres available in the movie dataset and identified movies with the highest number of matching genres. Movies with higher similarity scores were ranked higher in the recommendation list.

Sample Result:

User Preferences:
Action, Sci-Fi

Recommended Movies:

1. Avengers Endgame
   Matched Genres: Action, Sci-Fi
   Similarity Score: 2
   Match Percentage: 100%

2. Inception
   Matched Genres: Action, Sci-Fi
   Similarity Score: 2
   Match Percentage: 100%

3. Doctor Strange
   Matched Genres: Action, Sci-Fi
   Similarity Score: 2
   Match Percentage: 100%

Analysis:

• The system successfully generated personalized recommendations based on user interests.

• Similarity matching effectively identified movies that shared common genres with the user's preferences.

• The similarity score provided a simple and reliable method for ranking recommendations.

• The match percentage helped users understand the relevance of each recommendation.

• The recommendation history feature successfully stored previous recommendations and user ratings for future reference.

• The menu-driven interface improved user experience by providing options to generate recommendations, view history, clear history, and exit the application.

Overall, the system achieved its objective of providing personalized movie recommendations using content-based recommendation logic. The results demonstrate how user preferences can be effectively matched with item attributes to generate meaningful recommendations.

Screenshots:

Figure 1: Main Menu Interface

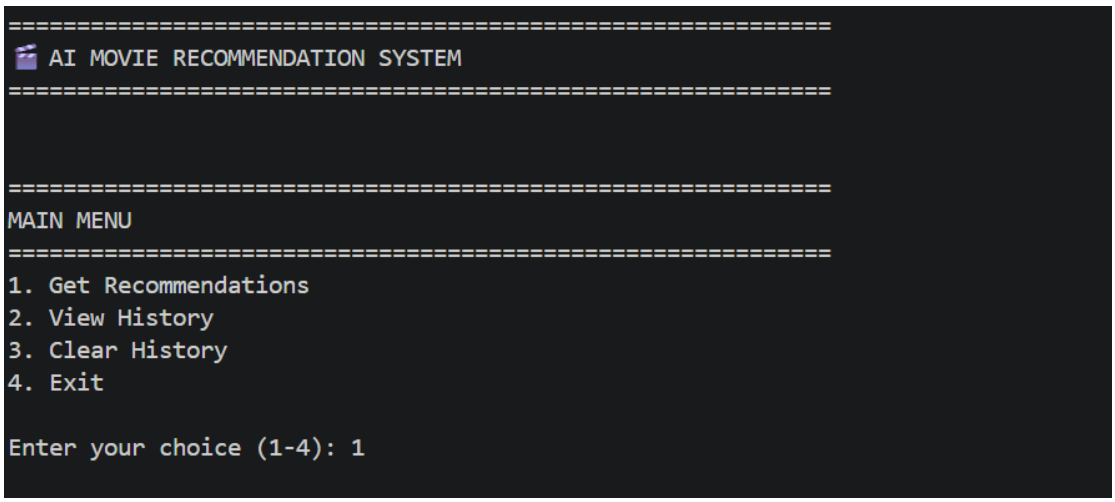


Figure 2: User Input Interface

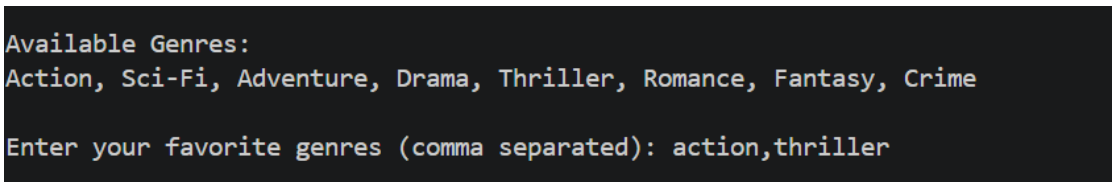


Figure 3: Movie Recommendations Output

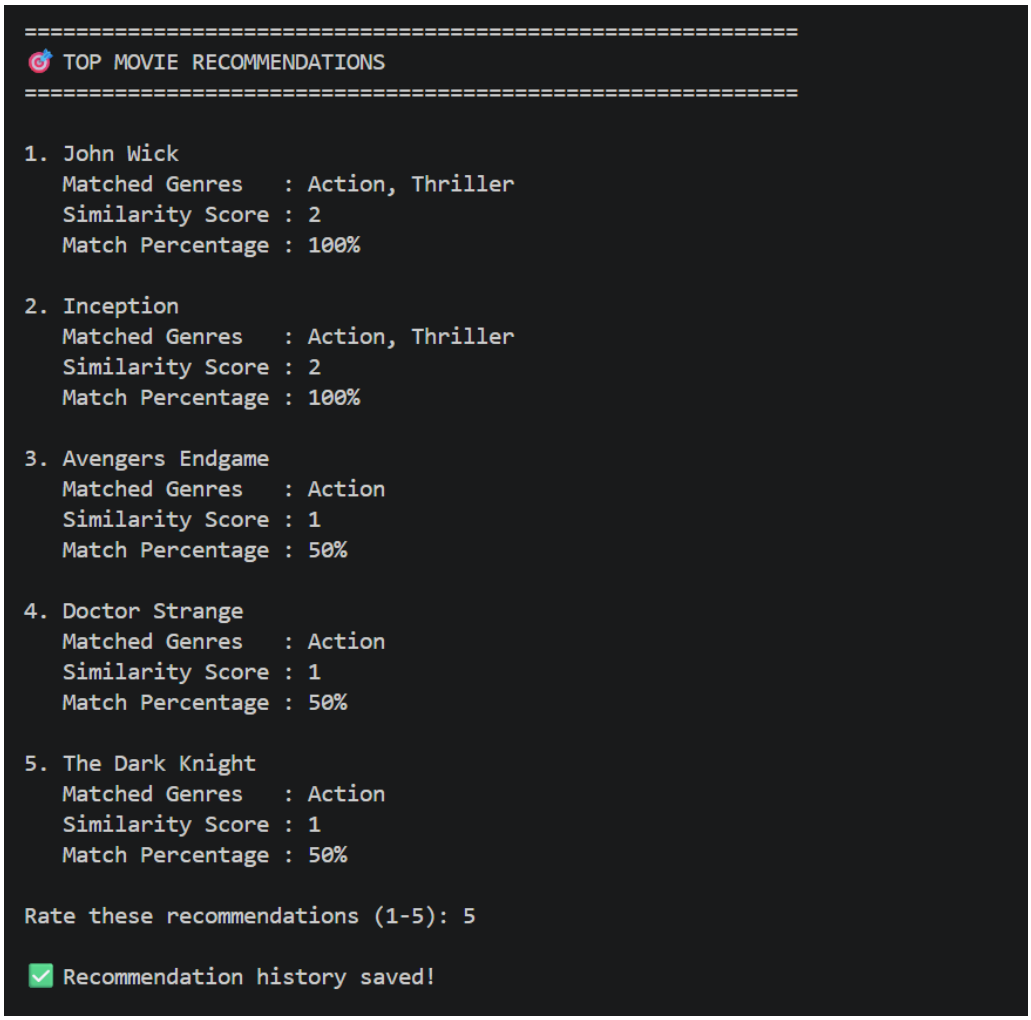
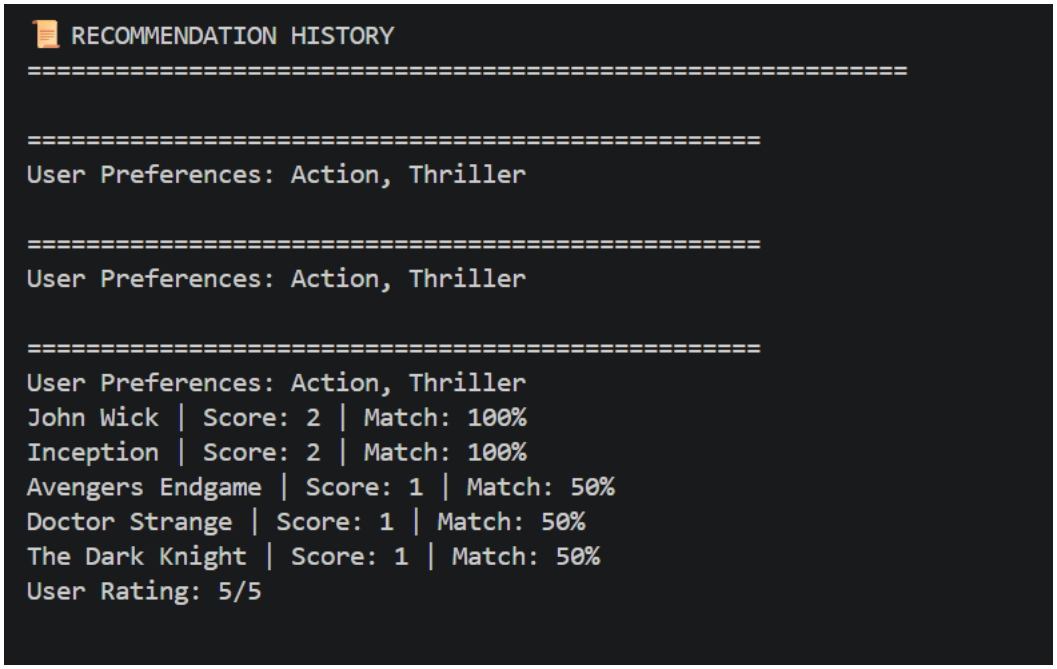


Figure 4: Recommendation History



CONCLUSION AND FUTURE SCOPE

Conclusion

The AI Movie Recommendation System was successfully developed using Python and achieved its objective of providing personalized movie recommendations based on user preferences. The system effectively collects user-selected genres, compares them with movie genres stored in the dataset, and generates relevant recommendations using similarity matching logic.

The implementation of similarity scores, match percentages, and matched genres helped improve the transparency and accuracy of the recommendation process. Additional features such as recommendation history management, viewing previous recommendations, and clearing stored history enhanced the overall usability of the application.

This project provided practical experience in Python programming, data structures, file handling, pattern matching, and recommendation system concepts. It also demonstrated how user preferences can be analyzed to generate meaningful and personalized suggestions.

Future Scope

The project can be further enhanced in several ways:

• Expand the movie database by including a larger collection of movies and genres.

• Store movie information in CSV files or databases for better scalability and management.

• Develop a graphical user interface (GUI) using Tkinter or PyQt for improved user interaction.

• Implement machine learning algorithms to generate more accurate and intelligent recommendations.

• Introduce user accounts and profiles to provide personalized recommendations based on user history.

• Integrate online movie databases and APIs to fetch real-time movie information.

• Add advanced filtering options such as movie ratings, release year, language, and popularity.

• Develop a web-based version of the application using modern web technologies.

In conclusion, this project serves as a strong foundation for understanding recommendation systems and can be extended into a more advanced AI-powered recommendation platform in the future.